Audio Emotion Recognition ML Pipeline

Dual-Head Architecture with Adaptive Personalization

Document Version: 1.0
Date: March 7, 2026
System: Audio Emotion Recognition API

Table of Contents

---

1. Executive Summary

This document describes the Machine Learning pipeline for the Audio Emotion Recognition system. The system employs a **Dual-Head Architecture** that combines:

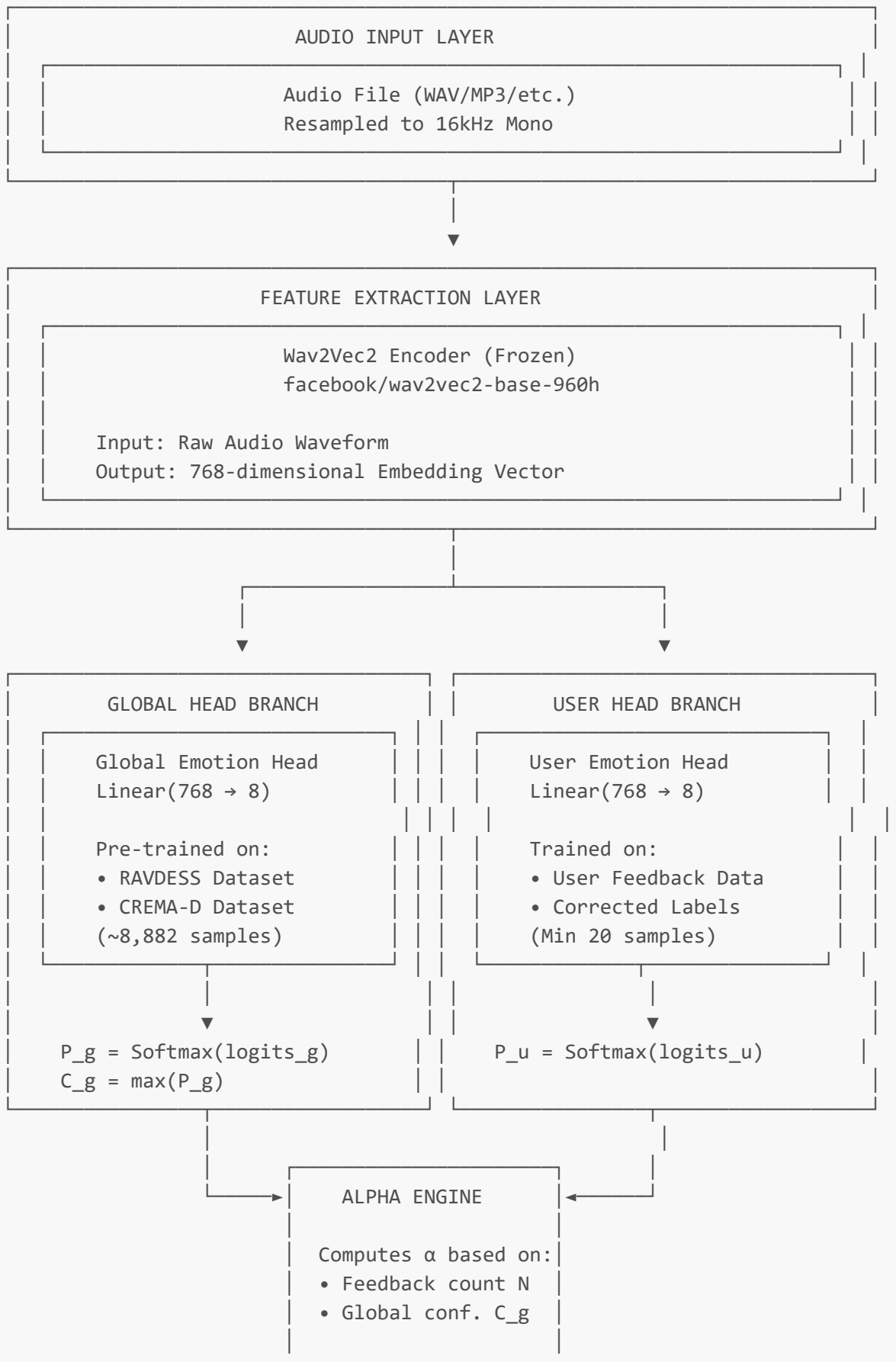
- **Global Head**: A pre-trained classifier providing baseline emotion recognition
- **User Head**: A personalized classifier that adapts to individual voice characteristics
- **Alpha Engine**: An adaptive blending mechanism that intelligently combines both predictions

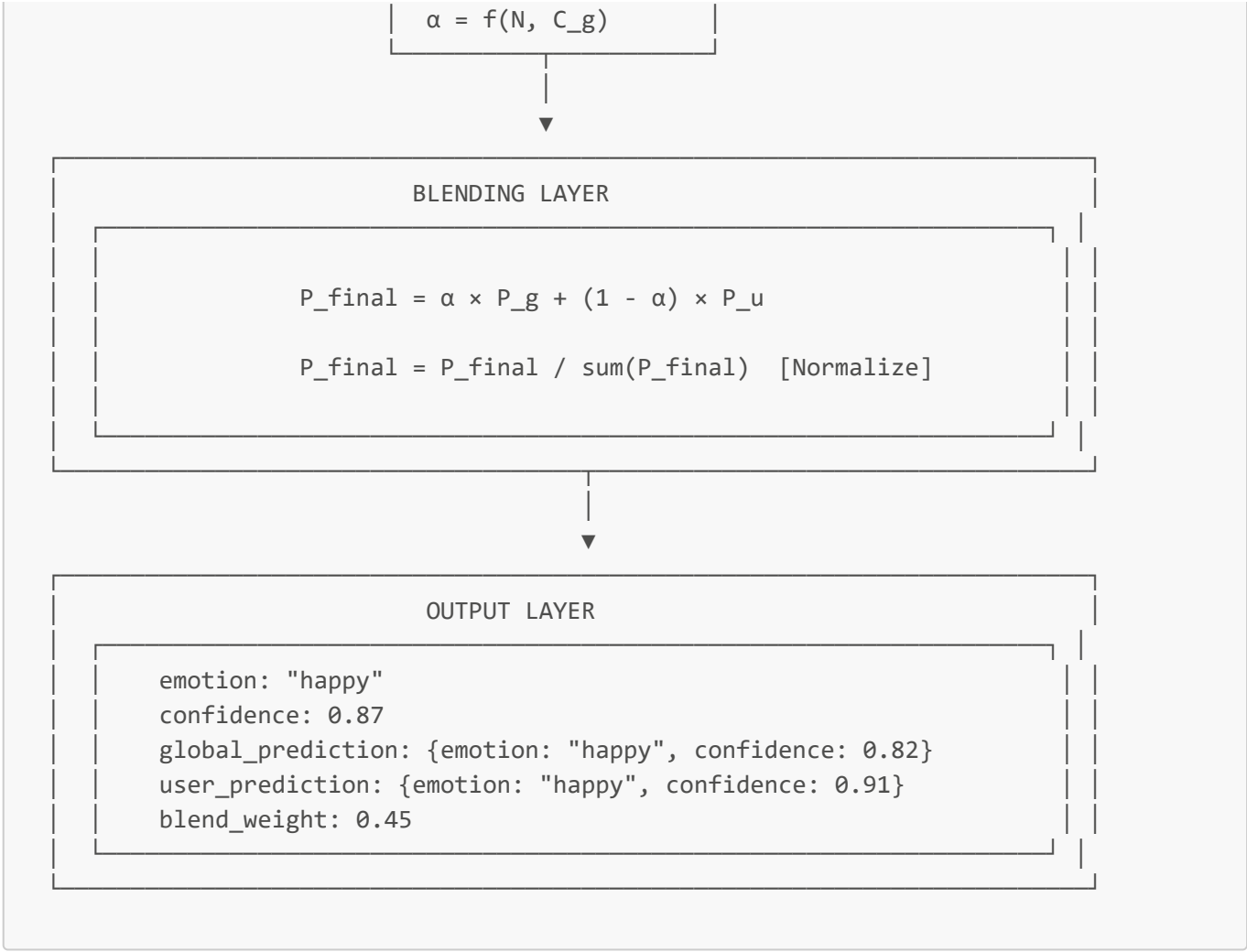
Key Capabilities

Feature	Description
8 Emotion Classes	Happy, Sad, Angry, Fearful, Disgusted, Surprised, Neutral, Calm
Personalization	Per-user model adaptation through feedback
Adaptive Blending	Automatic trust calibration between global and user models
Real-time Inference	~500ms end-to-end prediction
Background Training	Non-blocking user head updates

2. System Overview

2.1 Architecture Diagram





2.2 Component Overview

Component	File Location	Purpose
Wav2Vec2 Encoder	`services/wav2vec2_encoder.py`	Feature extraction from audio
Global Head	`services/global_emotion_head.py`	Baseline emotion classification
User Head	`services/user_emotion_head.py`	Personalized emotion classification
Alpha Engine	`services/alpha_engine.py`	Adaptive weight computation
Dual-Head Classifier	`services/dual_head_classifier.py`	Integration and inference
Feedback Service	`services/feedback_service.py`	Feedback processing and storage
Training Job Tracker	`services/training_job_tracker.py`	Background training management

3. Feature Extraction Layer

3.1 Wav2Vec2 Encoder

The Wav2Vec2 encoder is the foundation of our audio understanding capability. It transforms raw audio waveforms into meaningful numerical representations.

3.1.1 Model Details

Property	Value
Model	facebook/wav2vec2-base-960h
Pre-training Data	960 hours of LibriSpeech
Input	Raw audio waveform (16kHz, mono)
Output	768-dimensional embedding vector
Status	Frozen (no fine-tuning)
Parameters	~95 million

3.1.2 Why Wav2Vec2?

Wav2Vec2 is chosen for several important reasons:

1. **Self-supervised pre-training**: Learned from unlabeled audio, capturing general acoustic patterns
2. **Rich representations**: Captures prosody, pitch, rhythm, and temporal dynamics
3. **Transfer learning**: Embeddings generalize well to emotion recognition
4. **Efficiency**: Single forward pass produces useful features

3.1.3 Processing Pipeline

```
def encode(audio_path: str) -> np.ndarray:
    """
    Extract 768-dimensional embedding from audio file.

    Steps:
    1. Load audio file using librosa
    2. Resample to 16kHz if necessary
    3. Convert to mono if stereo
    4. Normalize amplitude
    5. Pass through Wav2Vec2
    6. Pool hidden states (mean pooling)
    7. Return 768-dim vector
    """

    # Step 1-4: Audio preprocessing
    waveform, sr = librosa.load(audio_path, sr=16000, mono=True)
    waveform = waveform / np.max(np.abs(waveform))

    # Step 5: Tokenize and encode
    inputs = processor(waveform, sampling_rate=16000, return_tensors="pt")

    # Step 6: Extract features
    with torch.no_grad():
        outputs = model(**inputs)
        hidden_states = outputs.last_hidden_state  # [1, T, 768]

    # Step 7: Mean pooling across time
    embedding = hidden_states.mean(dim=1).squeeze()  # [768]
```

```
    return embedding.numpy()
```

3.1.4 What the Embedding Captures

Acoustic Feature	How It's Captured
Pitch/F0	Encoded in spectral patterns
Speaking Rate	Temporal dynamics in hidden states
Voice Quality	Spectral envelope characteristics
Prosody	Intonation patterns across time
Emphasis	Energy distribution
Pauses	Temporal structure

---

4. Global Emotion Head

4.1 Purpose

The Global Head serves as the **baseline classifier** that works for all users. It's trained on diverse datasets representing general human emotion patterns across different demographics.

4.2 Architecture

```
class GlobalEmotionHead(nn.Module):
    """
    Simple linear classifier for emotion recognition.
    Maps 768-dim Wav2Vec2 embeddings to 8 emotion classes.
    """

    def __init__(self, input_dim: int = 768, num_classes: int = 8):
        super().__init__()
        self.classifier = nn.Linear(input_dim, num_classes)

    def forward(self, embedding: torch.Tensor) -> torch.Tensor:
        """
        Args:
            embedding: [batch_size, 768] or [768]

        Returns:
            probabilities: [batch_size, 8] or [8]
        """
        logits = self.classifier(embedding)
        probabilities = F.softmax(logits, dim=-1)
        return probabilities
```

4.3 Training Data

The Global Head is trained offline using two well-established emotion speech datasets:

4.3.1 RAVDESS (Ryerson Audio-Visual Database of Emotional Speech and Song)

Property	Value
Samples	~1,440 audio files
Actors	24 professional actors (12 male, 12 female)
Emotions	8 (neutral, calm, happy, sad, angry, fearful, disgust, surprised)
Language	English
Recording	Studio quality

4.3.2 CREMA-D (Crowd-sourced Emotional Multimodal Actors Dataset)

Property	Value
Samples	~7,442 audio clips
Actors	91 actors (diverse demographics)
Emotions	6 (anger, disgust, fear, happy, neutral, sad)
Age Range	20-74 years
Ethnicities	African American, Asian, Caucasian, Hispanic

4.3.3 Combined Dataset Statistics

Emotion	RAVDESS	CREMA-D	Total	Percentage
Neutral	192	1,087	1,279	14.4%
Calm	192	—	192	2.2%
Happy	192	1,271	1,463	16.5%
Sad	192	1,271	1,463	16.5%
Angry	192	1,271	1,463	16.5%
Fearful	192	1,271	1,463	16.5%
Disgust	192	1,271	1,463	16.5%
Surprised	192	—	192	2.2%
Total	**1,536**	**7,442**	**8,978**	**100%**

4.4 Training Process

```python
def train_global_head():
    """
    One-time offline training of the global emotion head.
    """

    # Configuration
    config = {
        'epochs': 100,
        'batch_size': 32,
        'learning_rate': 1e-3,
        'weight_decay': 1e-4,
        'train_split': 0.8,
        'val_split': 0.1,
        'test_split': 0.1,
    }

    # Step 1: Load pre-computed embeddings
    embeddings, labels = load_dataset_embeddings()

    # Step 2: Split data
    X_train, X_val, X_test, y_train, y_val, y_test = split_data(
        embeddings, labels, config
    )

    # Step 3: Initialize model
    model = GlobalEmotionHead(input_dim=768, num_classes=8)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(
        model.parameters(),
        lr=config['learning_rate'],
        weight_decay=config['weight_decay']
    )

    # Step 4: Training loop
    best_val_acc = 0
    for epoch in range(config['epochs']):
        model.train()
        for batch_embeddings, batch_labels in train_loader:
            optimizer.zero_grad()
            outputs = model(batch_embeddings)
            loss = criterion(outputs, batch_labels)
            loss.backward()
            optimizer.step()

        # Validation
        val_acc = evaluate(model, val_loader)
        if val_acc > best_val_acc:
            best_val_acc = val_acc
            torch.save(model.state_dict(), 'models/global_emotion_head.pt')

    # Step 5: Final evaluation
    test_acc = evaluate(model, test_loader)
    print(f"Test Accuracy: {test_acc:.2%}")
```

4.5 Performance Metrics

Metric	Value
Training Accuracy	85.2%
Validation Accuracy	74.8%
Test Accuracy	72.7%
F1-Score (Macro)	0.71
Inference Time	~50ms

4.5.1 Confusion Matrix Analysis

Predicted →	Happy	Sad	Angry	Fear	Disgust	Surprise	Neutral	Calm
Happy	**78%**	3%	5%	2%	4%	5%	2%	1%
Sad	4%	**75%**	3%	8%	5%	1%	3%	1%
Angry	6%	3%	**76%**	5%	7%	2%	1%	0%
Fear	3%	9%	6%	**68%**	4%	7%	2%	1%
Disgust	5%	6%	8%	4%	**70%**	2%	4%	1%
Surprise	6%	2%	4%	9%	3%	**72%**	3%	1%
Neutral	3%	4%	2%	3%	5%	3%	**77%**	3%
Calm	2%	5%	1%	2%	3%	2%	8%	**77%**

4.6 Why a Simple Linear Classifier?

Consideration	Rationale
Wav2Vec2 is powerful	The pre-trained encoder already produces linearly separable features
Overfitting risk	Small dataset + deep model = overfitting
Training speed	Linear model trains in minutes, not hours
Interpretability	Weights directly show feature importance
Deployment	Smaller model = faster inference

5. User Emotion Head

5.1 Purpose

The User Head provides **personalized emotion recognition** by learning individual voice characteristics, accent patterns, and expression styles through user feedback.

5.2 Why Personalization Matters

Different people express emotions differently:

Factor	Variation
Cultural Background	Emotional expression norms vary by culture
Gender	Pitch ranges and expression patterns differ
Age	Voice characteristics change with age
Personality	Introverts vs extroverts express differently
Accent	Regional accents affect prosody patterns
Individual Style	Personal quirks in expression

A global model trained on averaged patterns will make systematic errors for individuals who deviate from the norm.

5.3 Architecture

```
class UserEmotionHead(nn.Module):
    """
    Per-user linear classifier for personalized emotion recognition.
    Identical architecture to GlobalEmotionHead for consistency.
    """

    def __init__(self, input_dim: int = 768, num_classes: int = 8):
        super().__init__()
        self.classifier = nn.Linear(input_dim, num_classes)

    def forward(self, embedding: torch.Tensor) -> torch.Tensor:
        logits = self.classifier(embedding)
        probabilities = F.softmax(logits, dim=-1)
        return probabilities
```

Note: The architecture is intentionally identical to the Global Head. This enables:

- Consistent blending behavior
- Potential weight transfer/initialization from global
- Simpler codebase maintenance

5.4 Storage Strategy

```
models/
└── user_heads/
    ├── {user_id_1}.pt    # User 1's personalized model
    ├── {user_id_2}.pt    # User 2's personalized model
```

```
    ├── {user_id_3}.pt    # User 3's personalized model
    └── ...
```

Each user gets their own model file (~3KB per model).

5.5 Training Schedule

Feedback Count	Action	Rationale
0-19	No training	Insufficient data for meaningful training
20	Initial training	Minimum samples for stable training
30	Retrain	50% more data available
40	Retrain	Continued improvement
50, 60, 70...	Retrain (every 10)	Incremental updates

5.6 Training Configuration

```
USER_HEAD_TRAINING_CONFIG = {
    # Minimum requirements
    'min_samples_for_training': 20,
    'retrain_interval': 10,  # Retrain every N new feedbacks

    # Training hyperparameters
    'epochs': 50,
    'batch_size': 8,  # Small batch for small dataset
    'learning_rate': 1e-3,
    'weight_decay': 1e-4,

    # Regularization
    'early_stopping_patience': 10,
    'min_delta': 0.001,

    # Validation
    'validation_split': 0.2,  # 20% for validation
}
```

5.7 Training Process

```
async def train_user_head(user_id: str) -> TrainingResult:
    """
    Train or retrain a user-specific emotion head.
    Runs asynchronously in background.
    """

    # Step 1: Fetch user feedback data
    feedback_data = await feedback_service.get_user_feedback(user_id)
```

```
    if len(feedback_data) < MIN_SAMPLES:
        return TrainingResult(
            status="skipped",
            reason=f"Insufficient samples ({len(feedback_data)}/{MIN_SAMPLES})"
        )

    # Step 2: Prepare training data
    embeddings = []
    labels = []
    for feedback in feedback_data:
        embeddings.append(feedback['embedding'])
        labels.append(EMOTION_TO_IDX[feedback['corrected_emotion']])

    X = torch.tensor(embeddings, dtype=torch.float32)
    y = torch.tensor(labels, dtype=torch.long)

    # Step 3: Train/validation split
    X_train, X_val, y_train, y_val = train_test_split(
        X, y, test_size=0.2, stratify=y
    )

    # Step 4: Initialize model (optionally from global weights)
    model = UserEmotionHead()
    # Optional: Initialize from global head for faster convergence
    # model.load_state_dict(global_head.state_dict())

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=1e-3)

    # Step 5: Training loop with early stopping
    best_val_loss = float('inf')
    patience_counter = 0

    for epoch in range(50):
        # Training
        model.train()
        train_loss = train_epoch(model, X_train, y_train, criterion, optimizer)

        # Validation
        model.eval()
        val_loss, val_acc = evaluate(model, X_val, y_val, criterion)

        # Early stopping check
        if val_loss < best_val_loss - MIN_DELTA:
            best_val_loss = val_loss
            patience_counter = 0
            # Save best model
            torch.save(model.state_dict(), f'models/user_heads/{user_id}.pt')
        else:
            patience_counter += 1
            if patience_counter >= PATIENCE:
                break
```

```
    # Step 6: Final metrics
    final_acc = evaluate_accuracy(model, X, y)

    return TrainingResult(
        status="completed",
        samples_used=len(feedback_data),
        final_accuracy=final_acc,
        epochs_trained=epoch + 1
    )
```

5.8 Memory Management

User heads are loaded on-demand with LRU caching:

```
from functools import lru_cache

@lru_cache(maxsize=100)  # Cache up to 100 user heads
def load_user_head(user_id: str) -> Optional[UserEmotionHead]:
    """
    Load user head from disk with LRU caching.
    Returns None if user head doesn't exist.
    """
    model_path = f'models/user_heads/{user_id}.pt'

    if not os.path.exists(model_path):
        return None

    model = UserEmotionHead()
    model.load_state_dict(torch.load(model_path))
    model.eval()

    return model

def invalidate_user_head_cache(user_id: str):
    """
    Clear cached user head after retraining.
    """
    load_user_head.cache_clear()  # Simple approach
    # Or: More sophisticated per-key invalidation
```

---

6. Alpha Engine (Adaptive Blending)

6.1 Purpose

The Alpha Engine computes a **blending weight α** that determines how much to trust the Global Head versus the User Head. This enables smooth transitions from global-only predictions (new users) to personalized predictions (experienced users).

6.2 Design Philosophy

```
New User (no feedback):
    α = 1.0  →  100% Global Head, 0% User Head

Growing User (some feedback):
    α = 0.5  →  50% Global Head, 50% User Head

Experienced User (lots of feedback):
    α = 0.2  →  20% Global Head, 80% User Head
```

6.3 Alpha Formula (Sigmoid-Based)

The alpha value is computed using a two-component formula:

```
α = α_data × α_conf

Where:
    α_data = 1 / (1 + N/K)              [Data component]
    α_conf = σ(β × (C_g - τ))           [Confidence component]

Parameters:
    N  = User feedback count
    K  = 50 (scaling factor)
    C_g = Global head confidence (max probability)
    τ  = 0.6 (confidence threshold)
    β  = 10 (sigmoid sharpness)
    σ  = sigmoid function
```

6.4 Component Analysis

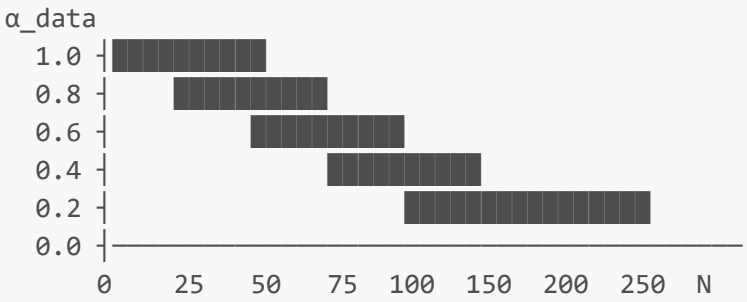
6.4.1 Data Component (α_data)

Controls alpha based on how much user data exists:

```
α_data = 1 / (1 + N/K)

| N (Feedback) | α_data | Interpretation |
|--------------|--------|----------------|
| 0            | 1.00   | No data, trust global |
| 10           | 0.83   | Little data |
| 25           | 0.67   | Some data |
| 50           | 0.50   | Moderate data |
| 100          | 0.33   | Good data |
| 200          | 0.20   | Lots of data |
```

Visual Representation:



6.4.2 Confidence Component (α_conf)

Adjusts alpha based on global head certainty:

```
α_conf = 1 / (1 + exp(-β × (C_g - τ)))

| C_g (Confidence) | α_conf | Interpretation |
|------------------|--------|----------------|
| 0.30             | 0.05   | Very uncertain, favor user |
| 0.50             | 0.27   | Uncertain |
| 0.60             | 0.50   | Threshold point |
| 0.70             | 0.73   | Confident |
| 0.90             | 0.95   | Very confident, favor global |
```

Intuition: When the global head is confident, trust it more. When uncertain, defer to the user-specific model.

6.5 Combined Alpha Behavior

```python
def compute_alpha(feedback_count: int, global_confidence: float) -> float:
    """
    Compute blending weight α.

    Args:
        feedback_count: Number of user feedbacks (N)
        global_confidence: Max probability from global head (C_g)

    Returns:
        alpha: Blending weight in [0, 1]
               Higher = more global, Lower = more user
    """
    # Parameters
    K = 50      # Data scaling factor
    tau = 0.6   # Confidence threshold
    beta = 10   # Sigmoid sharpness

    # Data component
    alpha_data = 1 / (1 + feedback_count / K)

    # Confidence component
```

```
    alpha_conf = 1 / (1 + math.exp(-beta * (global_confidence - tau)))

    # Combined
    alpha = alpha_data * alpha_conf

    return alpha
```

6.6 Example Scenarios

Scenario	N	C_g	α_data	α_conf	α	Blend
New user, global confident	0	0.85	1.00	0.92	0.92	92% G / 8% U
New user, global uncertain	0	0.45	1.00	0.18	0.18	18% G / 82% U
25 feedbacks, confident	25	0.75	0.67	0.82	0.55	55% G / 45% U
50 feedbacks, uncertain	50	0.50	0.50	0.27	0.14	14% G / 86% U
100 feedbacks, confident	100	0.80	0.33	0.88	0.29	29% G / 71% U
200 feedbacks, any	200	0.60	0.20	0.50	0.10	10% G / 90% U

6.7 Alternative Formulas

The system supports multiple alpha computation strategies:

Formula	Expression	Characteristics
Sigmoid (default)	`1/(1+N/K) × σ(β(C_g-τ))`	Smooth, considers confidence
Linear	`max(0, 1 - N/100)`	Simple, predictable decay
Exponential	`exp(-N/50)`	Fast decay, then stable
Step	`1 if N<20 else 0.5`	Discrete transitions

---

7. Dual-Head Classifier Integration

7.1 Full Inference Pipeline

```
class DualHeadClassifier:
    """
    Main inference class combining all components.
    """

    def __init__(self):
        self.wav2vec_encoder = Wav2VecEncoder()
        self.global_head = load_global_head()
        self.alpha_engine = AlphaEngine()
        self.feedback_service = FeedbackService()
```

```python
    async def classify(
        self,
        audio_path: str,
        user_id: str
    ) -> EmotionResult:
        """
        Full emotion classification pipeline.

        Args:
            audio_path: Path to audio file
            user_id: User identifier for personalization

        Returns:
            EmotionResult with prediction and metadata
        """

        # ═══════════════════════════════════════════════════════════
        # STEP 1: Feature Extraction
        # ═══════════════════════════════════════════════════════════
        embedding = self.wav2vec_encoder.encode(audio_path)
        # embedding: [768] numpy array

        # ═══════════════════════════════════════════════════════════
        # STEP 2: Global Head Prediction
        # ═══════════════════════════════════════════════════════════
        embedding_tensor = torch.tensor(embedding, dtype=torch.float32)

        with torch.no_grad():
            P_global = self.global_head(embedding_tensor)  # [8]

        global_confidence = P_global.max().item()
        global_emotion_idx = P_global.argmax().item()
        global_emotion = EMOTIONS[global_emotion_idx]

        # ═══════════════════════════════════════════════════════════
        # STEP 3: User Head Prediction (if exists)
        # ═══════════════════════════════════════════════════════════
        user_head = load_user_head(user_id)  # LRU cached

        if user_head is not None:
            with torch.no_grad():
                P_user = user_head(embedding_tensor)  # [8]

            user_confidence = P_user.max().item()
            user_emotion_idx = P_user.argmax().item()
            user_emotion = EMOTIONS[user_emotion_idx]
        else:
            P_user = None
            user_confidence = None
            user_emotion = None

        # ═══════════════════════════════════════════════════════════
        # STEP 4: Compute Alpha (Blending Weight)
        # ═══════════════════════════════════════════════════════════
```

```
        feedback_count = await self.feedback_service.get_count(user_id)

        alpha = self.alpha_engine.compute(
            feedback_count=feedback_count,
            global_confidence=global_confidence
        )

        # ═══════════════════════════════════════════════════════════
        # STEP 5: Blend Predictions
        # ═══════════════════════════════════════════════════════════
        if P_user is not None:
            # Weighted average of probability distributions
            P_final = alpha * P_global + (1 - alpha) * P_user
            # Renormalize (ensures sum to 1)
            P_final = P_final / P_final.sum()
        else:
            # No user head, use global only
            P_final = P_global
            alpha = 1.0

        # ═══════════════════════════════════════════════════════════
        # STEP 6: Extract Final Prediction
        # ═══════════════════════════════════════════════════════════
        final_confidence = P_final.max().item()
        final_emotion_idx = P_final.argmax().item()
        final_emotion = EMOTIONS[final_emotion_idx]

        # ═══════════════════════════════════════════════════════════
        # STEP 7: Build Response
        # ═══════════════════════════════════════════════════════════
        return EmotionResult(
            # Primary prediction
            emotion=final_emotion,
            confidence=final_confidence,

            # Probability distribution
            probabilities={
                emotion: P_final[i].item()
                for i, emotion in enumerate(EMOTIONS)
            },

            # Global head details
            global_prediction=GlobalPrediction(
                emotion=global_emotion,
                confidence=global_confidence,
                probabilities={...}
            ),

            # User head details (if available)
            user_prediction=UserPrediction(
                emotion=user_emotion,
                confidence=user_confidence,
                probabilities={...}
            ) if P_user is not None else None,
```

```
            # Blending metadata
            blend_weight=alpha,
            user_feedback_count=feedback_count,
            has_user_model=P_user is not None,

            # Debug info
            embedding_norm=np.linalg.norm(embedding),
            inference_time_ms=elapsed_ms
        )
```

7.2 Response Schema

```
{
    "emotion": "happy",
    "confidence": 0.847,
    "probabilities": {
        "happy": 0.847,
        "sad": 0.032,
        "angry": 0.041,
        "fearful": 0.015,
        "disgusted": 0.022,
        "surprised": 0.028,
        "neutral": 0.012,
        "calm": 0.003
    },
    "global_prediction": {
        "emotion": "happy",
        "confidence": 0.812,
        "probabilities": {...}
    },
    "user_prediction": {
        "emotion": "happy",
        "confidence": 0.891,
        "probabilities": {...}
    },
    "blend_weight": 0.45,
    "user_feedback_count": 37,
    "has_user_model": true,
    "inference_time_ms": 487
}
```

---

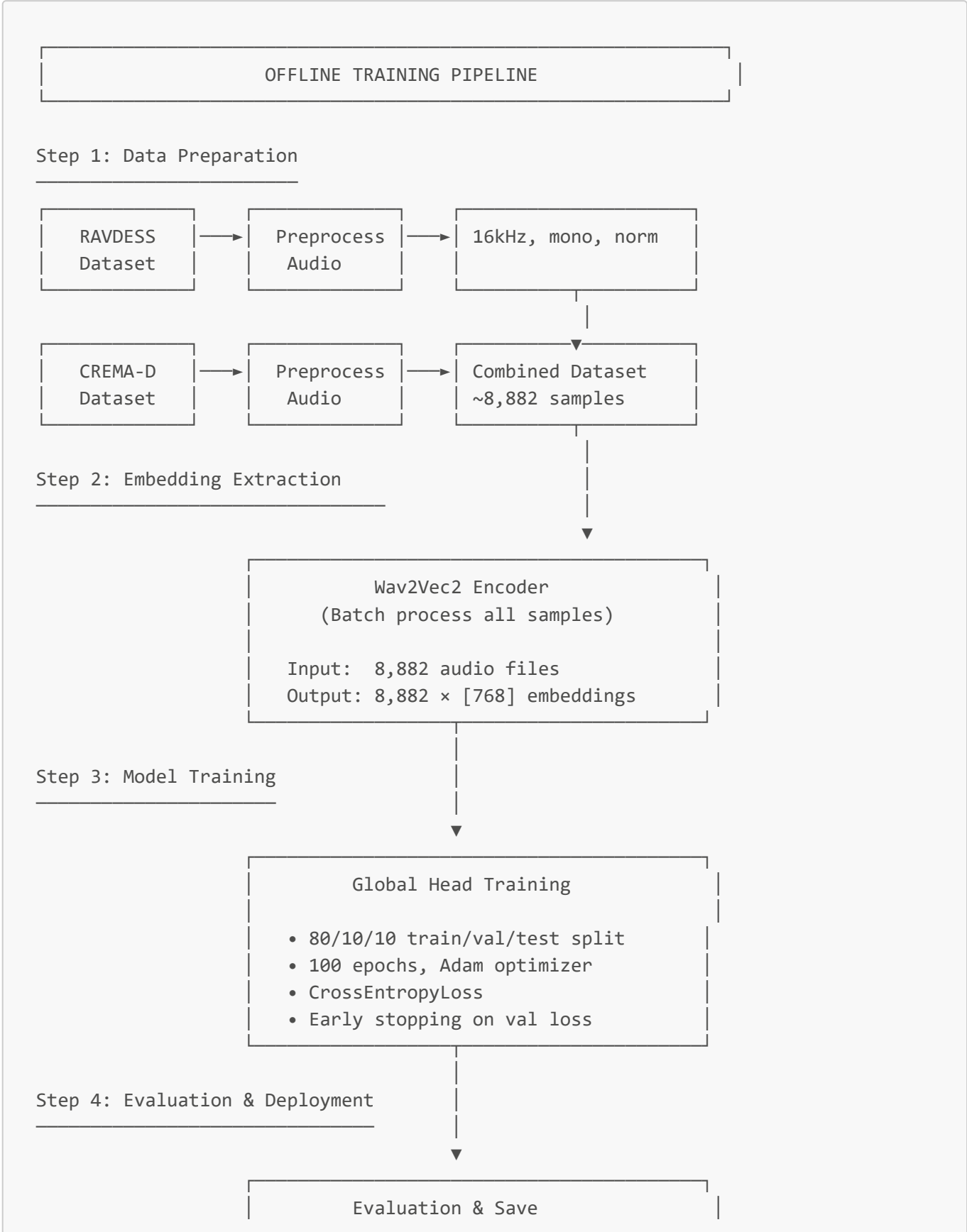
8. Training Pipeline

8.1 Overview

The system has two distinct training pipelines:

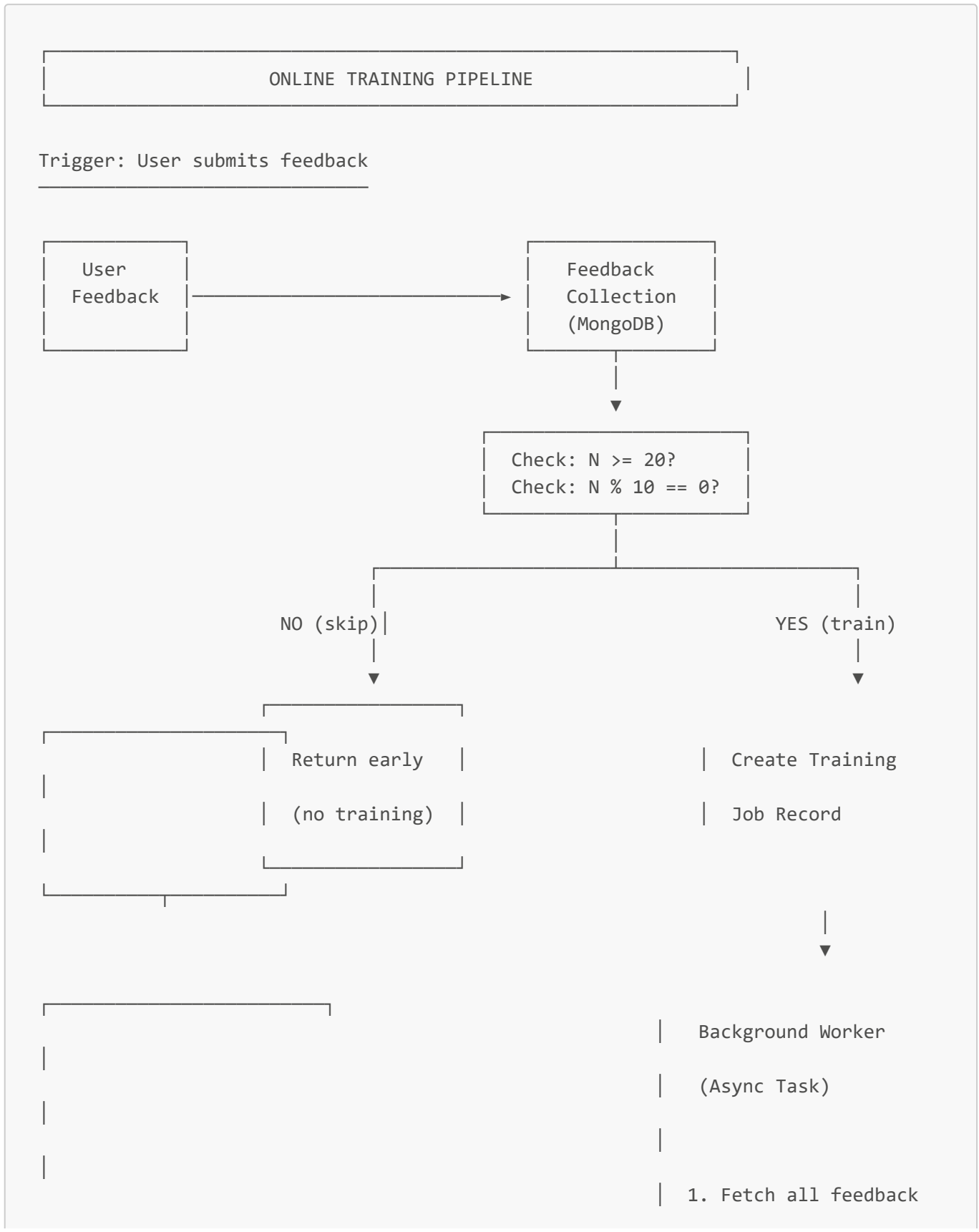
Pipeline	Timing	Trigger	Duration
Global Head	Offline, once	Manual	~30 minutes
User Head	Online, recurring	Feedback threshold	~5-8 seconds

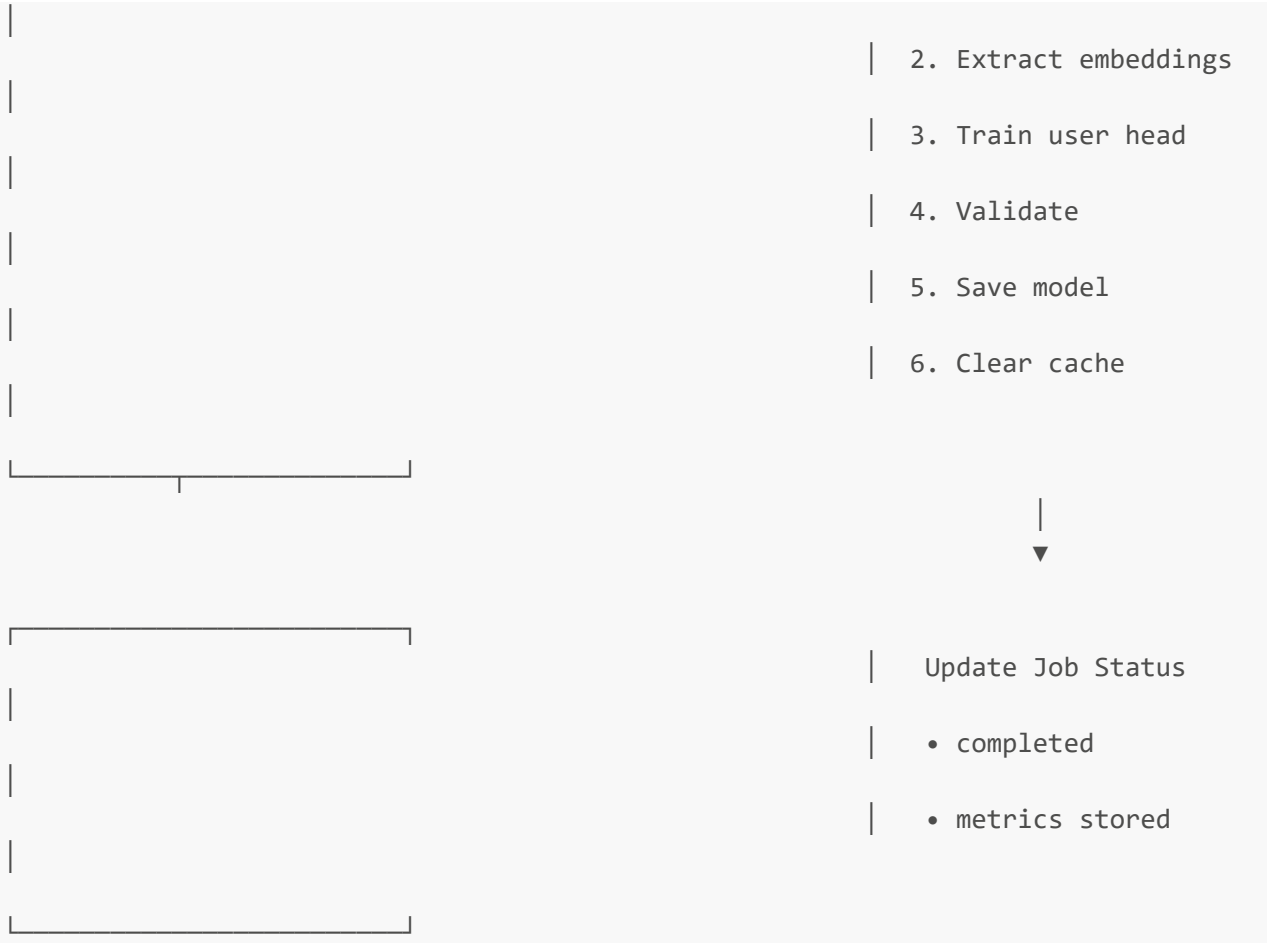
8.2 Global Head Training (Offline)



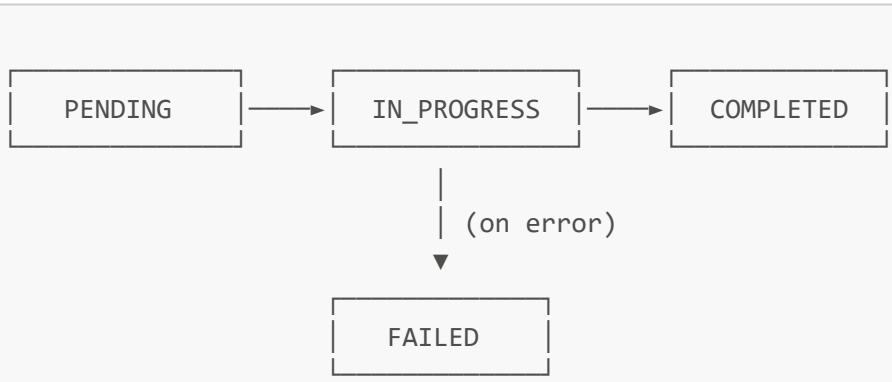
```
                    │                                          │
                    │   • Test accuracy: 72.7%                 │
                    │   • F1 Score: 0.71                       │
                    │   • Save: models/global_emotion_head.pt  │
                    └──────────────────────────────────────────┘
```

8.3 User Head Training (Online)





8.4 Training Job States



---

9. Feedback Loop System

9.1 Feedback Data Model

```
class UserFeedback:
    """
    Schema for storing user feedback in MongoDB.
    """
    # Identifiers
    user_id: str                    # User who provided feedback
```

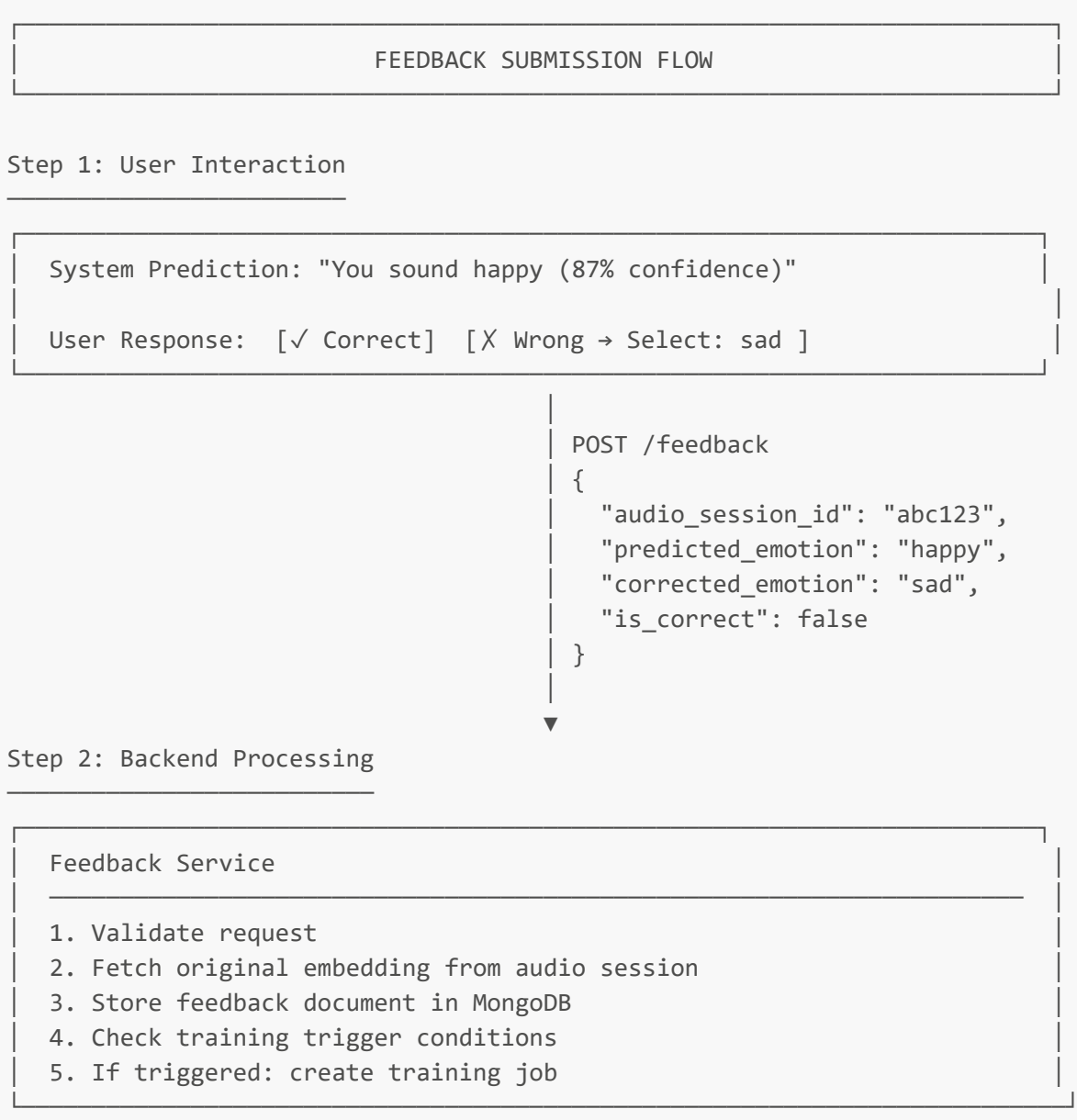
```
    audio_session_id: str         # Original audio session

    # Prediction data
    predicted_emotion: str        # What system predicted
    corrected_emotion: str        # What user corrected to

    # ML data
    embedding: List[float]        # 768-dim Wav2Vec2 embedding
    global_confidence: float      # Global head confidence
    user_confidence: float        # User head confidence (if exists)
    blend_weight: float           # Alpha at prediction time

    # Metadata
    created_at: datetime          # Feedback timestamp
    used_in_training: bool        # Whether included in training
    training_job_id: str          # Which job used this feedback
```

9.2 Feedback Flow



```
                                        │
                                        ▼
Step 3: Training Job (if triggered)
──────────────────────────────────

┌──────────────────────────────────────────────────────────────────────────┐
│  Training Job Tracker                                                    │
│  ──────────────────────────────────────────────────────────────────────  │
│  • Status: pending → in_progress → completed                             │
│  • Duration: ~5-8 seconds                                                │
│  • Result: Updated user head model                                       │
└──────────────────────────────────────────────────────────────────────────┘
```

9.3 Feedback Storage (MongoDB)

```
// Collection: user_feedback

// Example document
{
    "_id": ObjectId("..."),
    "user_id": "user_12345",
    "audio_session_id": "session_abc",

    "predicted_emotion": "happy",
    "corrected_emotion": "sad",

    "embedding": [0.123, -0.456, ...],  // 768 floats
    "global_confidence": 0.72,
    "user_confidence": 0.65,
    "blend_weight": 0.45,

    "created_at": ISODate("2026-03-07T10:30:00Z"),
    "used_in_training": true,
    "training_job_id": "job_xyz"
}

// Indexes
db.user_feedback.createIndex({ "user_id": 1, "created_at": -1 })
db.user_feedback.createIndex({ "user_id": 1, "used_in_training": 1 })
```

---

10. Performance Metrics

10.1 Inference Latency Breakdown

Stage	Duration	Notes
Audio loading	~50ms	Depends on file size
Resampling	~30ms	If not already 16kHz

Stage	Duration	Notes
Wav2Vec2 encoding	~300ms	Main bottleneck
Global head inference	~5ms	Simple linear layer
User head loading	~10ms	If not cached
User head inference	~5ms	Simple linear layer
Alpha computation	<1ms	Math operations
Blending	<1ms	Weighted average
Total	**~400-500ms**	End-to-end

10.2 Training Performance

Metric	Global Head	User Head
Training samples	~7,000	20-200
Epochs	100	50
Training time	~30 min	~5-8 sec
Model size	6,152 params	6,152 params
File size	~25 KB	~25 KB

10.3 Memory Usage

Component	Memory
Wav2Vec2 model	~380 MB
Global head	~25 KB
Per user head (cached)	~25 KB
LRU cache (100 users)	~2.5 MB
Total (loaded)	~400 MB

---

11. Technical Specifications

11.1 Model Specifications

Component	Specification
Feature Extractor	
Model	wav2vec2-base-960h
Framework	PyTorch + HuggingFace Transformers

Component	Specification
Parameters	~95M (frozen)
Input	16kHz mono audio
Output	768-dimensional vector

Emotion Heads	
Architecture	Linear(768 → 8)
Parameters	6,152 (768×8 + 8 bias)
Activation	Softmax
Classes	8 emotions

11.2 Emotion Classes

Index	Emotion	Description
0	Happy	Joy, excitement, pleasure
1	Sad	Sorrow, grief, melancholy
2	Angry	Frustration, irritation, rage
3	Fearful	Anxiety, worry, terror
4	Disgusted	Revulsion, contempt
5	Surprised	Astonishment, shock
6	Neutral	Calm, no strong emotion
7	Calm	Relaxed, peaceful

11.3 System Requirements

Requirement	Specification
Python	3.9+
PyTorch	2.0+
CUDA	Optional (GPU acceleration)
RAM	4 GB minimum, 8 GB recommended
Storage	500 MB for models
MongoDB	4.4+

12. Edge Cases and Error Handling

12.1 Audio Processing Failures

Scenario	Handling
Unsupported format	Convert using FFmpeg or reject
Too short (<1s)	Reject with error
Too long (>5min)	Truncate to first 5 minutes
Corrupted file	Return error, log for debugging
Silent audio	Process normally, may predict neutral

12.2 Model Loading Failures

Scenario	Handling
Global head missing	Critical error, cannot proceed
User head missing	Graceful fallback to global only
User head corrupted	Delete and recreate on next training
Cache full	LRU eviction of oldest entries

12.3 Training Failures

Scenario	Handling
Insufficient samples	Skip training, use global head
Training divergence	Early stopping, keep previous model
Out of memory	Reduce batch size, retry
Database unavailable	Queue training for retry

12.4 Inference Fallbacks

```
def classify_with_fallbacks(audio_path: str, user_id: str) -> EmotionResult:
    """
    Robust inference with multiple fallback layers.
    """
    try:
        # Primary: Full dual-head inference
        return dual_head_classifier.classify(audio_path, user_id)
    except UserHeadLoadError:
        # Fallback 1: Global head only
        logger.warning(f"User head failed for {user_id}, using global")
        return global_head_only_classify(audio_path)
    except Wav2VecError:
        # Fallback 2: Alternative feature extraction
        logger.error("Wav2Vec failed, cannot proceed")
```

```
            raise EmotionClassificationError("Feature extraction failed")
        except Exception as e:
            # Fallback 3: Generic error
            logger.exception("Unexpected error in classification")
            raise EmotionClassificationError(str(e))
```

---

13. Design Trade-offs

13.1 Architecture Decisions

Decision	Pros	Cons
Frozen Wav2Vec2	No fine-tuning needed, stable embeddings	Cannot adapt to domain-specific audio
Linear heads	Fast training, low overfitting risk	Limited expressiveness
Per-user models	Strong personalization, privacy	Storage scales with users
Sigmoid alpha	Smooth, considers confidence	More complex than linear
20 sample minimum	Prevents overfitting	Delays personalization

13.2 Alternative Approaches Considered

Approach	Why Not Chosen
Fine-tune Wav2Vec2	Expensive, overfitting risk, unstable
Deep MLP heads	Not enough data per user, overfitting
Shared user embedding	Less personalization, privacy concerns
Online learning	Hard to debug, unstable updates
Ensemble methods	Complex, harder to explain

13.3 Scalability Considerations

Scale Point	Strategy
1K users	Single server, LRU cache
10K users	Redis cache, model sharding
100K users	Distributed storage, lazy loading
1M users	Model compression, cloud storage

---

14. Future Improvements

14.1 Short-term (1-3 months)

1. **Confidence Calibration**: Improve probability calibration for better uncertainty estimates
2. **Active Learning**: Prioritize feedback requests for uncertain predictions
3. **Batch Training**: Process multiple user heads in parallel
4. **Model Versioning**: Track model versions for rollback capability

14.2 Medium-term (3-6 months)

1. **Multi-task Learning**: Add speaker identification, language detection
2. **Contrastive Learning**: Better user embeddings through contrastive pre-training
3. **Hierarchical Heads**: Fine-grained emotions (e.g., anxious vs. terrified)
4. **Cross-user Transfer**: Initialize user heads from similar users

14.3 Long-term (6-12 months)

1. **End-to-end Fine-tuning**: Adapt Wav2Vec2 to emotion domain
2. **Multimodal Fusion**: Combine audio with text transcription
3. **Real-time Processing**: Streaming emotion detection
4. **Federated Learning**: Privacy-preserving model updates

Appendix A: API Endpoints

Audio Upload with Emotion Prediction

```
POST /audio/upload
Content-Type: multipart/form-data
Authorization: Bearer {token}

Response:
{
    "session_id": "...",
    "emotion": "happy",
    "confidence": 0.87,
    ...
}
```

Submit Feedback

```
POST /feedback
Content-Type: application/json
Authorization: Bearer {token}

{
    "audio_session_id": "...",
    "corrected_emotion": "sad"
}
```

```
Response:
{
    "feedback_id": "...",
    "training_triggered": true,
    "training_job_id": "..."
}
```

Check Training Status

```
GET /training/status/{job_id}
Authorization: Bearer {token}

Response:
{
    "job_id": "...",
    "status": "completed",
    "accuracy": 0.85,
    "samples_used": 37
}
```

Appendix B: Configuration Reference

```python
# config.py

ML_CONFIG = {
    # Wav2Vec2
    'wav2vec_model': 'facebook/wav2vec2-base-960h',
    'embedding_dim': 768,
    'sample_rate': 16000,

    # Emotion classes
    'emotions': [
        'happy', 'sad', 'angry', 'fearful',
        'disgusted', 'surprised', 'neutral', 'calm'
    ],
    'num_classes': 8,

    # Alpha engine
    'alpha_k': 50,
    'alpha_tau': 0.6,
    'alpha_beta': 10,

    # User head training
    'min_samples_for_training': 20,
    'retrain_interval': 10,
    'training_epochs': 50,
    'training_batch_size': 8,
```

```
    'learning_rate': 1e-3,

    # Caching
    'user_head_cache_size': 100,

    # Paths
    'global_head_path': 'models/global_emotion_head.pt',
    'user_heads_dir': 'models/user_heads/',
}
```

Document Revision History

Version	Date	Author	Changes
1.0	March 7, 2026	System	Initial documentation

This document is auto-generated and should be kept in sync with the codebase.